Project summary

The project I selected is: `nanobot: Ultra-Lightweight Personal AI Assistant`

What I aim to reproduce:

Deploy and run the entire project successfully, and establish a stable connection between nanobot and Telegram (prerequisite for the whole experiment).

Based on the above experimental setup, I conducted the following experiment： Under the same task and environment, only the model was switched (via `.nanobot/config.json: agents.defaults.model/provider`). Through controlled A/B testing, we quantitatively compared the success rate, latency, tool invocation, and modification quality of two different agent models on the same task: **"developing and adding desktop screenshot capability on macOS and sending via Telegram"**. This allowed us to assess the reproducibility boundaries of the feature implementation and the capability differences between the models.

Setup notes

Environment

> macOS

- This is a moderately difficult development task, so I have restricted the task to the macOS system (rather than extending it to all platforms) to reduce the development intensity per unit time.

> git

- We need to use git worktree to create separate working directories.

> python 3.12

- The runtime environment for nanobot must be `python >= 3.11`.

> Telegram

- Telegram will be used to interact with nanobot to evaluate the feasibility of the newly added features.

Data

- Github Repo
  - https://github.com/xcosmosbox/nanobot
  - This is the repository I forked from the official nanobot repo, to be used as my code submission repository.
  - > **Important**
    >
    > To preserve the change records of both models, I have **not merged the branches modified by the two models into the main branch**.
    >
    > If you want to view my commit history in detail, please manually select the `minimax` or `deepseek` branch.

- Models
    - minimax coding plan
        - Minimax-M2.5
    - DeepSeek API
        - DeepSeek-chat

Reproduction target

Under the same task and environment, only the model was switched (via `.nanobot/config.json: agents.defaults.model/provider`). Through controlled A/B testing, we quantitatively compared the success rate, latency, tool invocation, and modification quality of two different agent models on the same task: **"developing and adding desktop screenshot capability on macOS and sending via Telegram"**. This allowed us to assess the reproducibility boundaries of the feature implementation and the capability differences between the models.

- Task: Add `ScreenshotTool (macOS only)` and support sending to users
- Control method: Only switch `agents.defaults.model/provider`
- Experiment scale: 2 agents
- Goal: Observe the actual performance of two different models under the nanobot project, evaluate their performance using additional metrics, and verify the convenience and generality of the nanobot framework.

Metric definition

This experiment uses the following metrics to evaluate performance:

1. **Successful completion of the long-chain development task**: Whether each model can fully implement the moderately difficult, moderately time-consuming task.
2. **Total time taken for the entire development task**: Development efficiency of the two models under the same framework and environment for the same task.
3. **Total number of tool invocations**: Whether the models used more concise invocation patterns during development.
4. **Number of invocations per tool**: Distribution of tool usage.
5. **Number of passed/failed code functionality tests**: Pass rate of unit test files independently developed by the models.
6. **Scale of code changes**: Whether code modifications are concise, efficient, free of redundancy, and within scope.
7. **Actual runtime performance (manual verification)**: Final judgment of whether the implemented function works correctly.

Modification

The core modifications of this experiment are as follows:

Without changing the experimental task definition and environment configuration, only the following A/B agent configurations were switched.

- Agent A
  - model: deepseek-chat
  - provider: deepseek
- Agent B
  - model: MiniMax-M2.5
  - provider: minimax

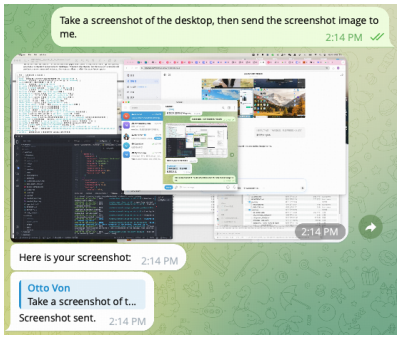
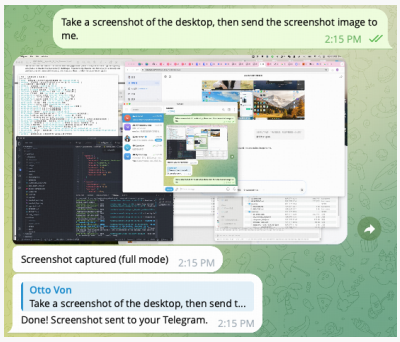
All other conditions remained consistent:

- Identical code baseline: `@mian-244650235c6f24739f3fe7390111d6c1f26439cb`
- Identical task prompt: `task_prompt.txt`
- Identical runtime environment

Results

Data Source: Log files from the two executions, and JSON conversions of the session files from the two executions (session files are in JSONL format and need to be converted to JSON). The log files and JSON files have been placed in their respective branches.

Core Results Overview

	Dev task Completion	Total Time	Total Tool Calls	Per-tool Calls	Unit Test	Code Change Size	Runtime Result
Minimax M2.5	DONE	468.64s	83	{read_file:20, edit_file:24, write_file:4, list_dir:4, exec:31}	ALL PASS	`+320` / `-4`	OK 
DeepSeek V3.2	DONE	701.55s	52	{read_file:10, edit_file:20, write_file:2, list_dir:7, exec:13}	ALL PASS	`+388` / `-4`	OK 

Tool Call and Modification Quality Analysis

Tool Call Analysis

1. Minimax M2.5: 83 calls, taking 468.64 seconds.
2. DeepSeek V3.2: 52 calls, taking 701.55 seconds.
3. Average time per call: ~5.65s/call for Minimax, ~13.49s/call for DeepSeek.
4. Minimax had a significantly higher number of `exec` calls (31 vs 13), indicating a stronger reliance on an iterative verification approach of "modify a bit - run - modify again", with an overall more aggressive style.
5. DeepSeek had a higher proportion of `list_dir` calls (7 vs 4), favoring an exploratory path and making one-time modifications only after sufficient information is obtained.

Code Modification Quality Analysis

1. The instructions implemented by both models are essentially equivalent, with unit test results and runtime performance also being basically the same. Additionally, the scale of code changes is comparable between the two.
2. At the same development cost, Minimax can complete the task with fewer modifications, reducing token consumption. DeepSeek adopts a more conservative strategy, which may lead to redundant coverage and thus higher token consumption.

Summary

1. Minimax: Higher call density but sufficiently fast overall speed.
2. DeepSeek: Lower call frequency but relatively slow speed.

Debug diary

Separate worktrees are created to isolate code modifications by each agent, preventing contamination of the main branch. Additionally, modifications by each agent can be performed independently.

Solution： Use the command `git worktree add -b <your-branch-name> <worktree-path> main` to create two independent new branches from the main branch to distinguish the modifications between them.

Conclusions

The conclusions of this experiment are as follows:

1. Different agents exhibit different performance under the same environment and the same task.
2. Both Minimax M2.5 and DeepSeek V3.2 are able to complete this experiment.
3. No non-reproducible issues are found. The nanobot system is successfully deployed and fully configured, and can interact via Telegram.